

Iris Keras Classifier

Dataset Iris, rete neurale Sequential e risultati del modello

A cura di

Alisia Sallemi
Emanuele Brivio

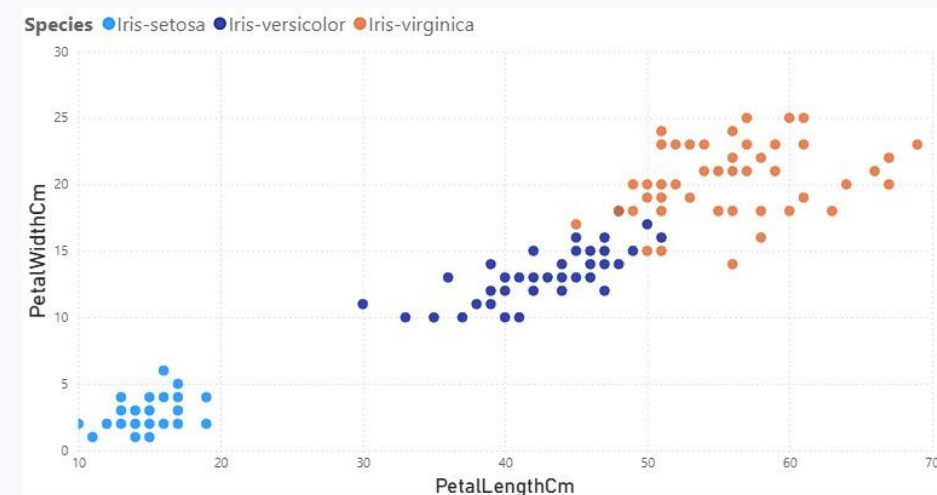
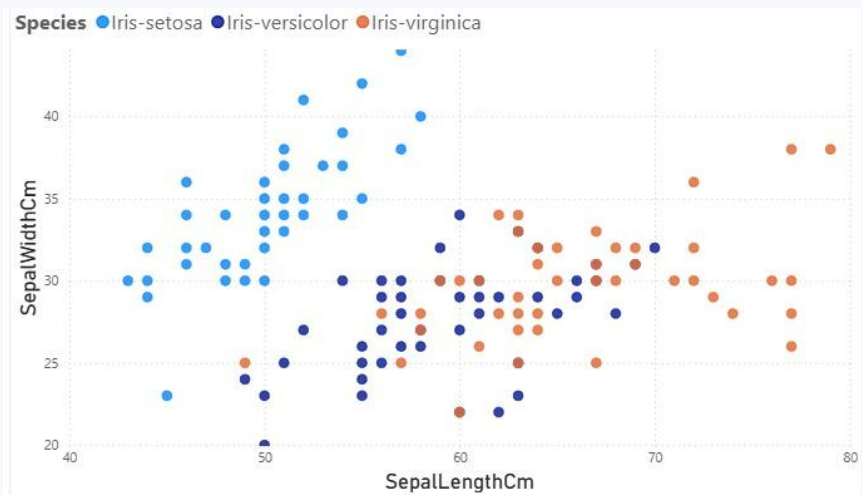
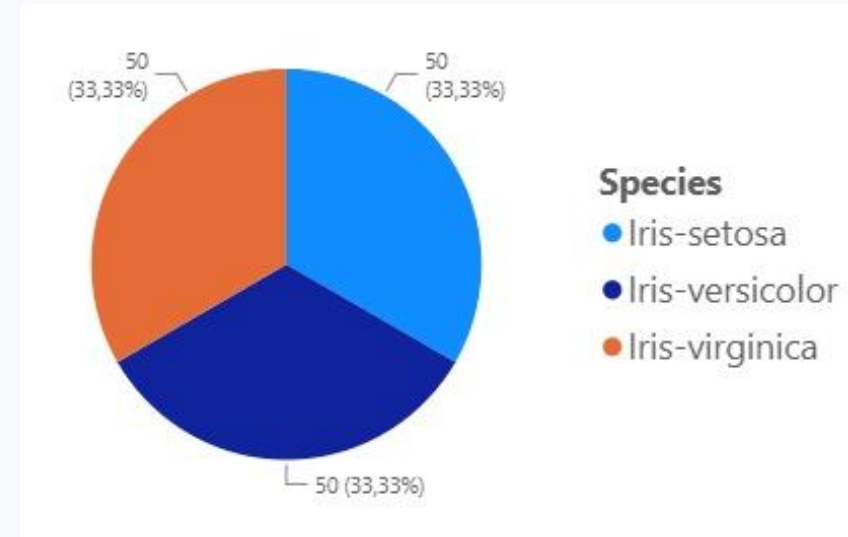
<https://github.com/asallemi1/keras>

Dataset Iris

150 osservazioni, 4 feature numeriche e una target a 3 classi bilanciate.

Struttura

- Righe: 150
- Colonne: Id, SepalLengthCm, SepalWidthCm, PetalLengthCm, PetalWidthCm, Species
- Target: Species
- Classi: setosa, versicolor, virginica



Preprocessing e split

Le trasformazioni evitano leakage e rendono coerente il training con il test.

1

Feature

Rimosse Id e Species;
restano 4 colonne
numeriche.

2

Label encoding

Species -> 0, 1, 2 con
LabelEncoder.

3

StandardScaler

Fit sul training set,
transform sul test set.

4

Train/val/test

70/15/15 stratificato.
Train: 105
Validation: 22
Test: 23

Parametri di training

I parametri principali determinano capacità del modello e dinamica di apprendimento.

Input	4 feature standardizzate
Layer hidden	Dense(16, relu) + Dense(8, relu)
Output	Dense(3, softmax)
Loss	sparse_categorical_crossentropy
Optimizer	Adam, learning_rate = 0.001
Batch size	32
Epochs	200



Sparse Categorical Crossentropy: loss function utilizzata per problemi di classificazione multiclasse. Confronta le etichette reali con le probabilità predette dal modello senza richiedere la codifica one-hot encoding.



Adam Optimizer: algoritmo di ottimizzazione che aggiorna automaticamente i pesi della rete combinando i vantaggi di Momentum e RMSProp, garantendo una convergenza rapida ed efficiente.



Batch Size: numero di campioni elaborati dal modello prima di aggiornare i pesi; batch più grandi rendono l'addestramento più stabile, mentre batch più piccoli richiedono meno memoria.



Epochs: numero di volte in cui l'intero dataset di addestramento viene utilizzato dal modello; un numero adeguato di epoche permette di apprendere i pattern senza incorrere in overfitting.



Momentum: tiene conto degli aggiornamenti precedenti dei pesi, creando una sorta di “inerzia” che accelera la discesa verso il minimo della loss function e riduce le oscillazioni.

RMSProp: adatta automaticamente il tasso di apprendimento per ogni parametro in base all'entità dei gradienti recenti, rendendo l'addestramento più stabile ed efficiente.

Summary del modello

Rete Sequential fully connected: 4 input, due layer hidden e output softmax a 3 classi.

```
Model: "sequential"
```

Layer	Output Shape	Param #
dense (Dense)	(None, 16)	80
dense_1 (Dense)	(None, 8)	136
dense_2 (Dense)	(None, 3)	27

```
Trainable params: 243
```

```
Optimizer params: 488
```

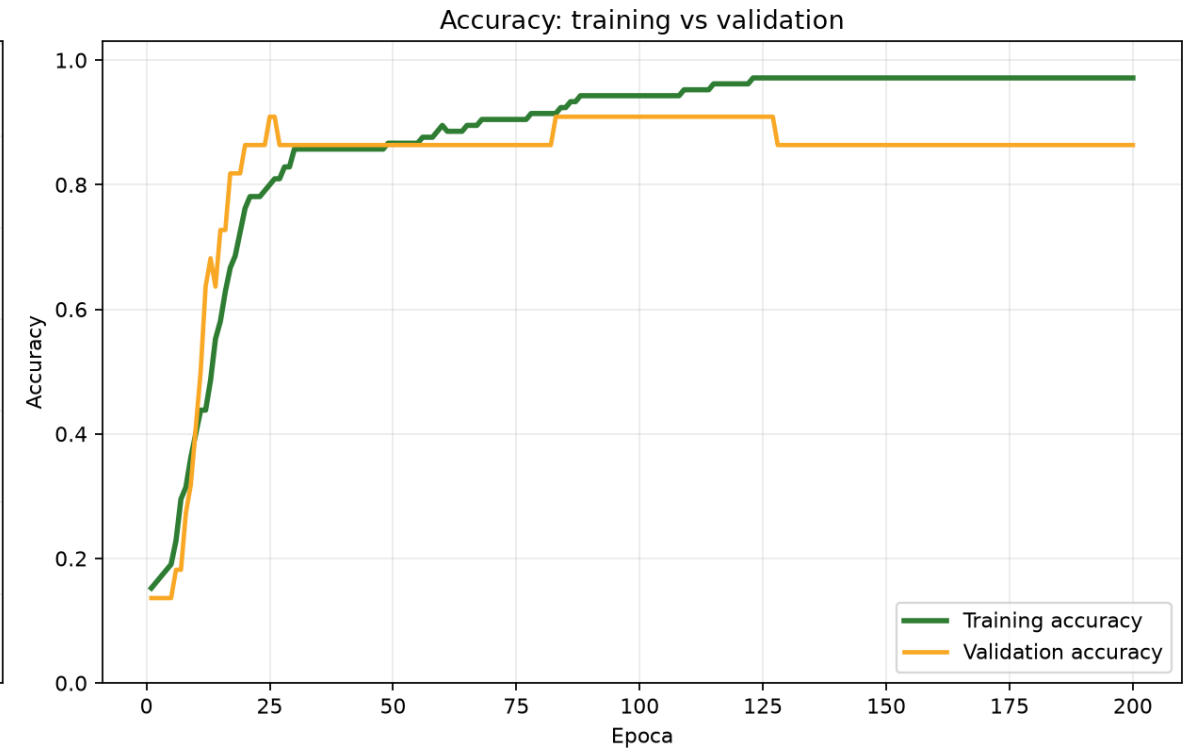
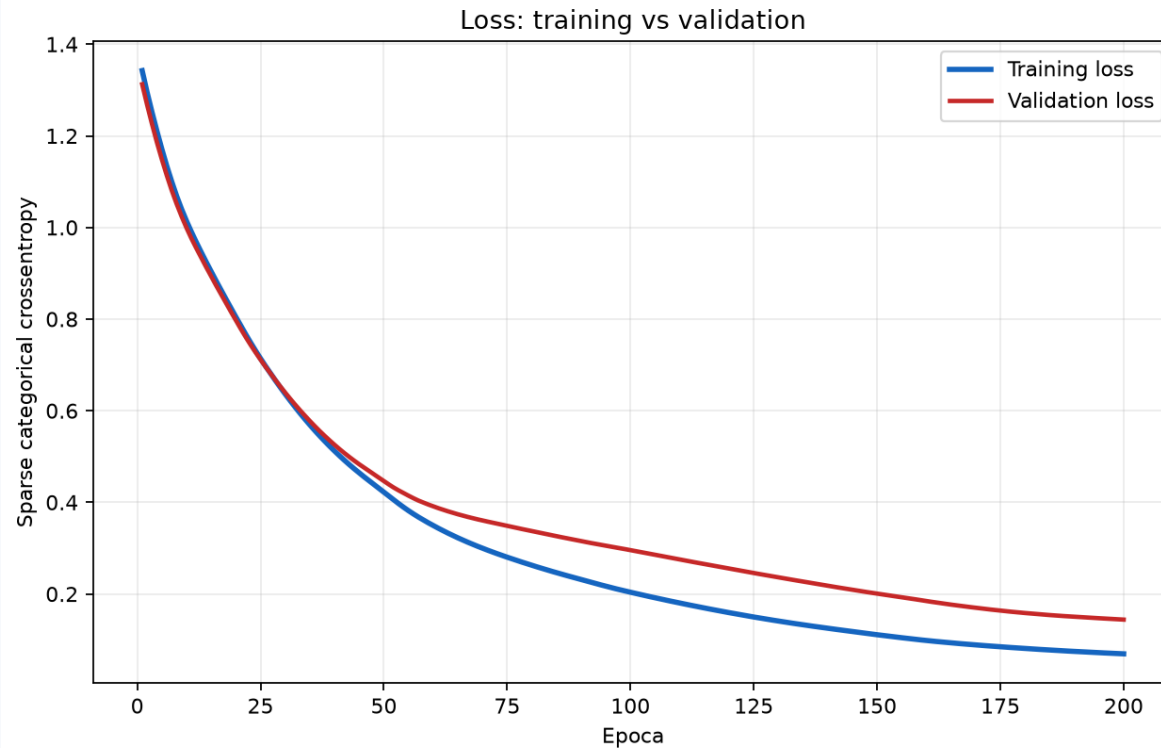
```
Total params: 731
```

Come leggere i parametri

- Dense(16): 4×16 pesi + 16 bias = 80
- Dense(8): 16×8 pesi + 8 bias = 136
- Output: 8×3 pesi + 3 bias = 27

History di loss e accuracy

Questi grafici sono calcolati su training e validation sets.



Ultima epoca: Train Accuracy = 97,1%, Validation Accuracy = 86,4%.

Plateau raggiunto intorno alla 100^a epoca.

Con **Early Stopping Callback**, il training si sarebbe arrestato automaticamente a quel punto.

Risultati sul test set

La valutazione finale usa il 15% del dataset tenuto fuori dal training e dal validation.

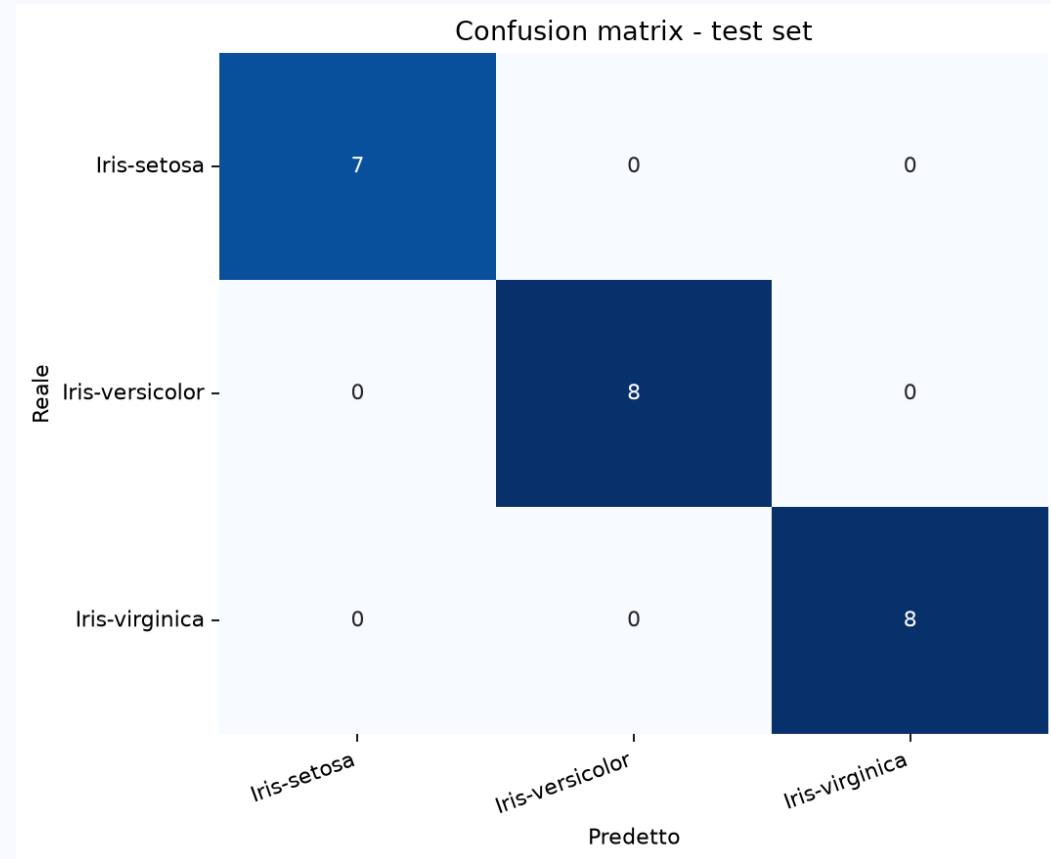
Accuracy

100%

Loss

0.114

Interpretazione: 23 classificazioni corrette su 23 nel test set. Validation accuracy finale: 86.4%.



Sviluppi futuri

Prossimi passi per rendere il training più robusto.

ReduceLROnPlateau

Riduce il learning rate quando la validation loss si stabilizza davvero.

Dropout

Spegne casualmente una porzione di neuroni durante il training per ridurre overfitting.

EarlyStopping

Interrompe il training quando non ci sono miglioramenti per un certo numero di epoche.

Regolarizzazione L1/L2

Penalizza pesi troppo grandi; L1 favorisce sparsità, L2 stabilizza i pesi.